

Lab 2: Layered Architecture Design (Logical View)

This lab focuses on designing the Layered Architecture for a specific Microservice within the HRM system (focusing on the Leave Service here), representing the static structure and relationships between the system's components. This is a crucial step to clarify the data flow before diving into detailed coding (Lab 3).

Objectives

1. Understand the principles and constraints of the Layered Architecture Pattern.
2. Define the 4 main layers and their responsibilities in the Backend system.
3. Identify the key components within each layer for the Leave Management module.
4. Model the logical view using a UML Component Diagram.

Tool & Technology Setup

Tool	Purpose	Installation/Setup Guide
draw.io (Diagrams.net)	Creating professional UML Component Diagrams.	Access online via a web browser (no installation required) or download the desktop app.
Whiteboard/Pen & Paper	Brainstorming and initial sketching of layers/components.	Standard study materials.

Activity Practice 1: Defining Layers and Responsibilities

Goal: Formally define the 4 layers and their roles within the structure of a Backend Service (leave_service).

Step-by-Step Instructions:

1. **Define the 4 Layers:** Create a table documenting the name and primary purpose of each layer, strictly adhering to the Layered Rule (A layer can only communicate with the layer immediately below it).

Layer	Purpose / Responsibility	Output / Artifact
-------	--------------------------	-------------------

1. Presentation Layer (Routing)	Handles HTTP requests (REST APIs) from the React App, extracts the JWT Token, checks permissions (RBAC), and returns an HTTP response (JSON).	Routers / Controllers (e.g., LeaveRouter, LeaveController)
2. Business Logic Layer	Contains core business rules: Calculating leave balances, checking limits (no more than 3 consecutive days, max 5 people/day), and routing Real-time alerts.	Services / Managers (e.g., LeaveService)
3. Persistence Layer (Data Access)	Responsible for mapping business objects into database queries (PostgreSQL) and executing CRUD operations. Writes Audit Logs.	Repositories / DAOs (e.g., LeaveRepository, LeaveLogDAO)
4. Data Layer (Physical Storage)	The physical database storage system.	Database Schema (PostgreSQL: leave_requests, MongoDB: LeaveLogs)

2. **Identify Data Flow:** Sketch a typical request flow when an employee submits a new Leave Request.

- *Action:* Trace the flow: Client Request (React) $\rightarrow$ Layer 1 (Router receives request) $\rightarrow$ Layer 2 (Check leave balance/Company rules) $\rightarrow$ Layer 3 (Insert data) $\rightarrow$ Layer 4 (PostgreSQL saves) $\rightarrow$ Layer 3 $\rightarrow$ Layer 2 (Trigger Socket.io) $\rightarrow$ Layer 1 $\rightarrow$ Client Response (Success message).

Activity Practice 2: Component Identification (Leave Management Module)

Goal: Break down the "Create Leave Request" feature into specific software components residing in the top 3 layers.

Step-by-Step Instructions:

1. **Identify Components:** For the "Submit Leave Request" feature, identify the specific software components (modules or files) that will exist in Layers 1, 2, and 3.
 - **Layer 1 (Presentation):**
 - *Component Name:* LeaveController
 - *Responsibility:* Receives the POST /api/leave_ser request, extracts user_id, reason, start_date, total_days from req.body. Calls the Business Logic Layer and returns a 400 error or a 200 JSON response.
 - **Layer 2 (Business Logic):**
 - *Component Name:* LeaveService
 - *Responsibility:* Receives data, enforces company rules: (1) Checks if total_days > 3. (2) Calls Layer 3 to verify if usedDays + total_days > maxDays. (3) Checks the limit of 5 people taking leave on the same day. Emits Socket.io events.
 - **Layer 3 (Persistence):**
 - *Component Name:* LeaveRepository
 - *Responsibility:* Translates requests into actual queries (e.g., INSERT INTO leave_requests... RETURNING *) and calls the LeaveLog.create(...) function to save logs in MongoDB.
2. **Define Interfaces:** Determine the primary interface (method signature) provided by the Business Logic Layer to the Presentation Layer, and by the Persistence Layer to the Business Logic Layer.
 - LeaveService Interface (provided to Layer 1): public Object createLeaveRequest(LeaveDTO data, UserToken currentUser)
 - LeaveRepository Interface (provided to Layer 2): public int countApprovedLeavesInDateRange(Date start, Date end), public LeaveEntity insertLeave(LearnEntity leave)

Activity Practice 3: Component Diagram Modeling

Goal: Create a visual model of the Layered Architecture using a UML Component Diagram.

Step-by-Step Instructions (Using draw.io):

1. **Draw Layers:** Create three large, stacked rectangles representing the **Presentation Layer**, **Business Logic Layer**, and **Persistence Layer**.
2. **Place Components:** Inside each layer, draw a UML Component shape (a rectangle with two small rectangles on the side) for the components identified in Practice 2 (LeaveController, LeaveService, LeaveRepository).
3. **Define Interfaces (Provided/Required):**
 - *Provided Interface:* Use a "lollipop" symbol (a circle on a line) attached to the LeaveService component, labeled ILeaveService. This shows what the component offers (e.g., leave calculation functions).
 - *Required Interface:* Use a "socket" symbol (a semi-circle on a line) attached to the LeaveController component, "plugging" into the ILeaveService lollipop. This shows that the Controller does not handle logic itself but relies on the Service.
 - *Repeat* this for the connection between LeaveService and LeaveRepository (ILeaveRepository).
4. **Enforce Strict Dependency:** Use dashed arrows to show the dependency flow. Ensure all arrows point strictly downward (Layer 1 $\rightarrow$ Layer 2 $\rightarrow$ Layer 3). This visually enforces the rule that layers cannot be bypassed (e.g., the Controller cannot write SQL queries directly but must go through the Service).